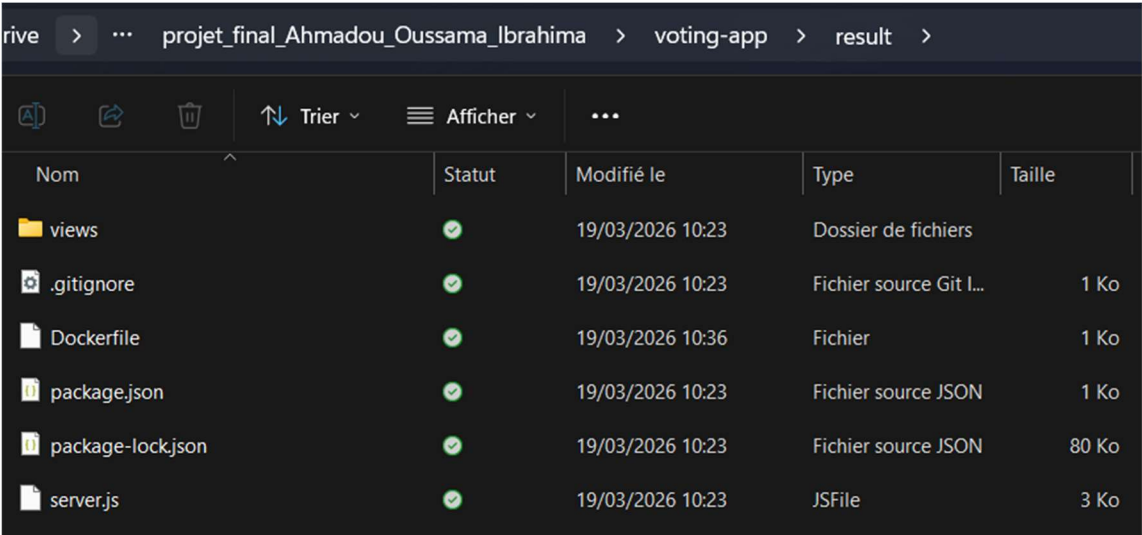


### Compte-rendu de Projet pDOKR3 :

Ce document détaille le processus de modernisation d'une application distribuée de vote. L'objectif de ce projet était de remplacer un lancement manuel via des scripts par une architecture entièrement conteneurisée avec Docker, orchestrée localement via Docker Compose, puis déployée en haute disponibilité sur un cluster Docker Swarm.

#### Étape 1 :

Pour commencer, nous avons conteneurisé les trois modules principaux de l'application (vote, worker, result) en créant un Dockerfile dédié pour chacun d'entre eux.



The screenshot shows a file explorer window with the path: `rive > ... > projet_final_Ahmadou_Oussama_Ibrahima > voting-app > result >`. The file list includes:

| Nom               | Statut | Modifié le       | Type                    | Taille |
|-------------------|--------|------------------|-------------------------|--------|
| views             | ✓      | 19/03/2026 10:23 | Dossier de fichiers     |        |
| .gitignore        | ✓      | 19/03/2026 10:23 | Fichier source Git I... | 1 Ko   |
| Dockerfile        | ✓      | 19/03/2026 10:36 | Fichier                 | 1 Ko   |
| package.json      | ✓      | 19/03/2026 10:23 | Fichier source JSON     | 1 Ko   |
| package-lock.json | ✓      | 19/03/2026 10:23 | Fichier source JSON     | 80 Ko  |
| server.js         | ✓      | 19/03/2026 10:23 | JSFile                  | 3 Ko   |

Drive > ... projet\_final\_Ahmadou\_Oussama\_Ibrahima > voting-app > vote >

| Nom              | Statut | Modifié le       | Type                    | Taille |
|------------------|--------|------------------|-------------------------|--------|
| static           | ✓      | 19/03/2026 10:23 | Dossier de fichiers     |        |
| templates        | ✓      | 19/03/2026 10:23 | Dossier de fichiers     |        |
| .gitignore       | ✓      | 19/03/2026 10:23 | Fichier source Git I... | 1 Ko   |
| app.py           | ✓      | 19/03/2026 10:23 | Fichier source Pyth...  | 2 Ko   |
| Dockerfile       | ✓      | 19/03/2026 10:37 | Fichier                 | 1 Ko   |
| requirements.txt | ✓      | 19/03/2026 10:23 | Document texte          | 1 Ko   |

Drive > ... projet\_final\_Ahmadou\_Oussama\_Ibrahima > voting-app > worker

| Nom           | Statut | Modifié le       | Type                    | Taille |
|---------------|--------|------------------|-------------------------|--------|
| .gitignore    | ✓      | 19/03/2026 10:23 | Fichier source Git I... | 1 Ko   |
| Dockerfile    | ✓      | 19/03/2026 10:38 | Fichier                 | 1 Ko   |
| Program.cs    | ✓      | 19/03/2026 10:23 | Fichier source C#       | 6 Ko   |
| Worker.csproj | ✓      | 19/03/2026 10:23 | Fichier source C# P...  | 1 Ko   |

Afin de respecter les bonnes pratiques, nous avons :

Utilisé les versions officielles et requises pour chaque environnement (Python 3.11, .NET 7, Node.js 18).

Privilégié les versions Alpine lorsque c'était possible afin de réduire drastiquement la taille de nos images et limiter la surface d'attaque.

Exposé uniquement les ports nécessaires au bon fonctionnement.

## Étape 2 :

L'application d'origine était conçue pour tourner en local, avec des composants cherchant à se connecter aux bases de données via l'adresse localhost. Dans une architecture conteneurisée, chaque conteneur possède sa propre interface réseau. Nous avons donc modifié le code source pour remplacer localhost par le nom des futurs services Docker.

De plus, puisque l'orchestration est désormais gérée par Docker, les anciens scripts de lancement bash sont devenus obsolètes. Nous les avons donc définitivement supprimés pour nettoyer l'arborescence du projet.

| Nom                      | Statut | Modifié le       | Type                    | Taille |
|--------------------------|--------|------------------|-------------------------|--------|
| result                   | ✓      | 19/03/2026 10:27 | Dossier de fichiers     |        |
| vote                     | ✓      | 19/03/2026 10:37 | Dossier de fichiers     |        |
| worker                   | ✓      | 19/03/2026 10:38 | Dossier de fichiers     |        |
| .gitignore               | ✓      | 19/03/2026 10:23 | Fichier source Git I... | 1 Ko   |
| docker-compose.swarm.yml | ✓      | 19/03/2026 11:26 | Fichier source Yaml     | 2 Ko   |
| docker-compose.yml       | ✓      | 19/03/2026 18:11 | Fichier source Yaml     | 2 Ko   |

## Étape 3 :

Pour regrouper et lancer simultanément tous les services (les 3 applications et les 2 bases de données), nous avons créé un fichier docker-compose.yml à la racine du projet.

Ce fichier respecte les points suivants :

Réseaux isolés : Création d'un front-tier et d'un back-tier

Persistance des données : Déclaration d'un volume nommé db-data monté sur /var/lib/postgresql/data pour ne perdre aucun vote lors d'un redémarrage.

Gestion des dépendances : Utilisation de la directive depends\_on couplée à des healthcheck (ex: pg\_isready pour PostgreSQL) pour garantir que les applications web ne démarrent que lorsque les bases de données sont opérationnelles.

Le lancement de la commande docker compose up -d --build s'exécute avec succès :

```
#41 8.568 Worker -> /source/bin/Release/net7.0/Worker.dll
#41 8.701 Worker -> /app/
#41 DONE 9.1s

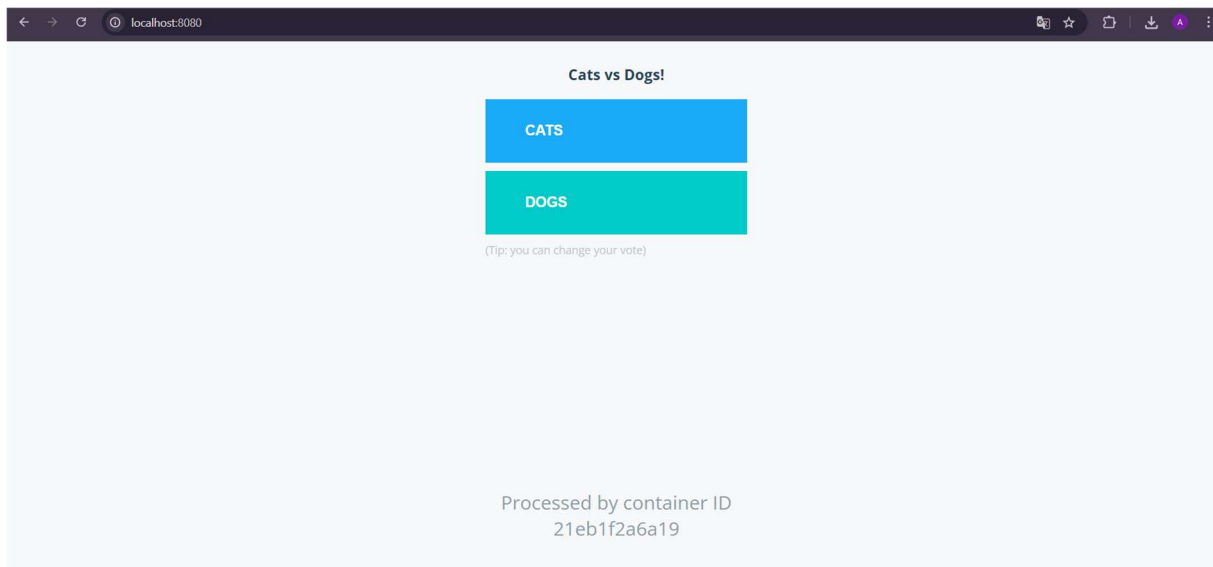
#42 [worker stage-1 4/5] COPY --from=build /app .
#42 DONE 0.2s

#43 [worker stage-1 5/5] RUN chown -R appuser:appgroup /app
#43 DONE 0.5s

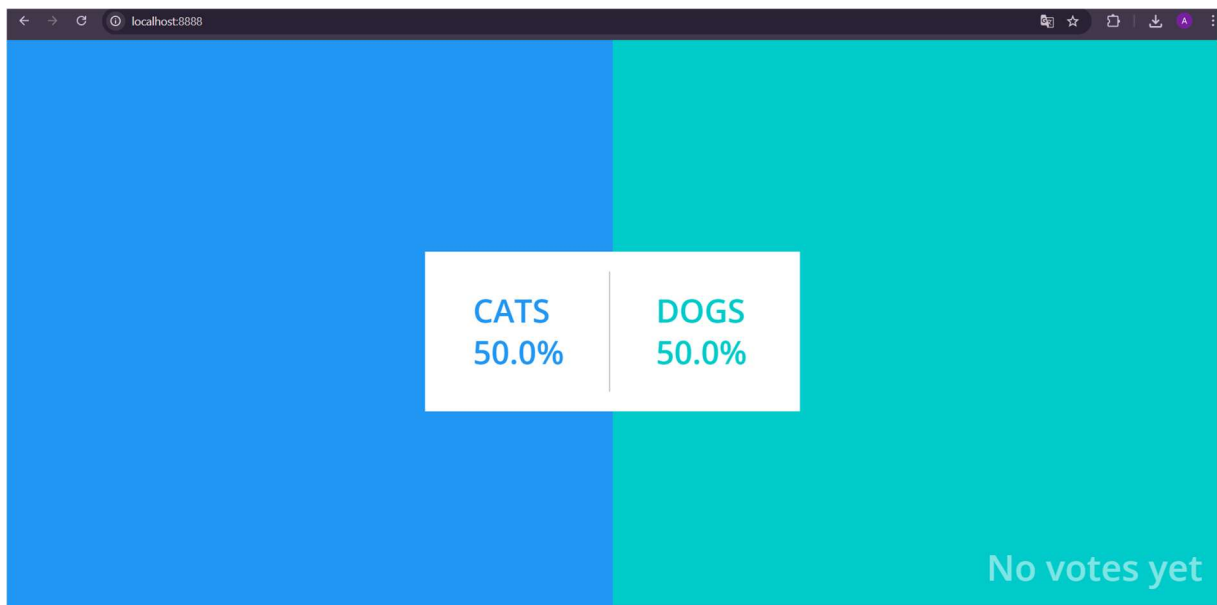
#44 [worker] exporting to image
#44 exporting layers
#44 exporting layers 0.6s done
#44 exporting manifest sha256:5b15e91f6f2d27458163581a39c354ab1d6bd85c9a6452346db30f6fc465f99e 0.0s done
#44 exporting config sha256:ff3137b494dabcea4f9ac408404c7435bcefb99816bd9191270e0677a5d55f4c 0.0s done
#44 exporting attestation manifest sha256:d8c233629c61936f0cb02e0e8721aef9279e2e29d4b9f52ef143a7282f11c69c 0.1s done
#44 exporting manifest list sha256:cb007de41ff388bef9af2969cd87e950a119b378041eab0f2c9ffb48e6dbb96c 0.0s done
#44 exporting manifest list sha256:cb007de41ff388bef9af2969cd87e950a119b378041eab0f2c9ffb48e6dbb96c 0.0s done
#44 naming to docker.io/library/voting-app-worker:latest done
#44 unpacking to docker.io/library/voting-app-worker:latest
#44 unpacking to docker.io/library/voting-app-worker:latest 0.3s done
#44 DONE 1.3s
[+] up 33/33
✓Image postgres:15-alpine Pulled 21.3s
✓Image redis:alpine Pulled 10.0s
✓Image voting-app-vote Built 60.7s
✓Image voting-app-result Built 60.7s
✓Image voting-app-worker Built 60.7s
✓Network voting-app_front-tier Created 0.2s
✓Network voting-app_back-tier Created 0.1s
✓Volume voting-app_db-data Created 0.0s
✓Container voting-app-redis-1 Healthy 8.1s
✓Container voting-app-db-1 Healthy 8.1s
✓Container voting-app-vote-1 Created 0.4s
✓Container voting-app-result-1 Created 0.4s
✓Container voting-app-worker-1 Created 0.4s
C:\Users\adial\OneDrive - SUPINFO\SUPINFO\pDOKR3 & 3DOKRd - Developing and Delivering Software with Docker\projet_final_Ahmadou_Oussama_Ibrahima\voting-app>
```

L'application est alors parfaitement fonctionnelle sur notre machine locale :

Page de vote



## Page des résultats



Les pages s'affichent et le vote fonctionne !

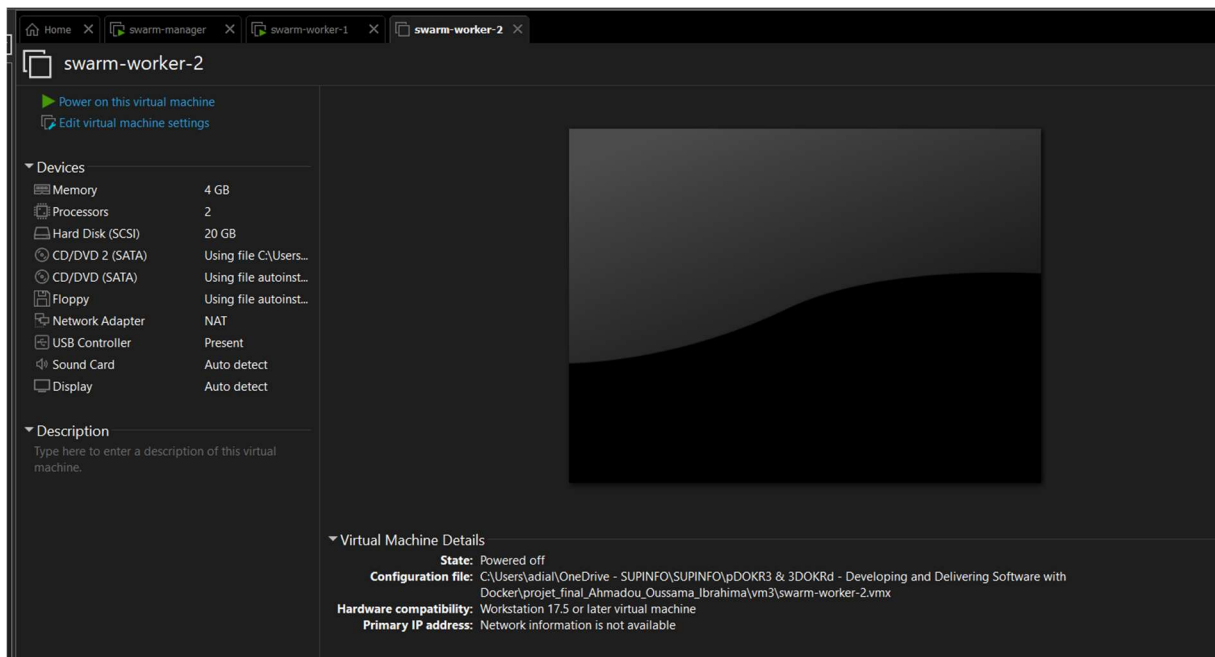
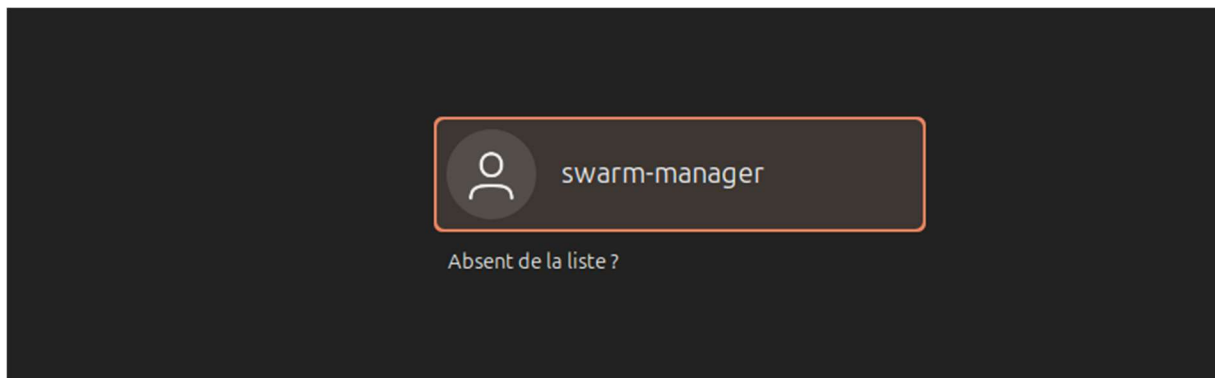
A screenshot of a web browser window at localhost:8080. The page has a light blue background. On the right side, there is a section titled 'Cats vs Dogs!'. It contains two buttons: a blue button labeled 'CATS' with a white checkmark icon, and a teal button labeled 'DOGS'. Below these buttons, a small tip reads '(Tip: you can change your vote)'.



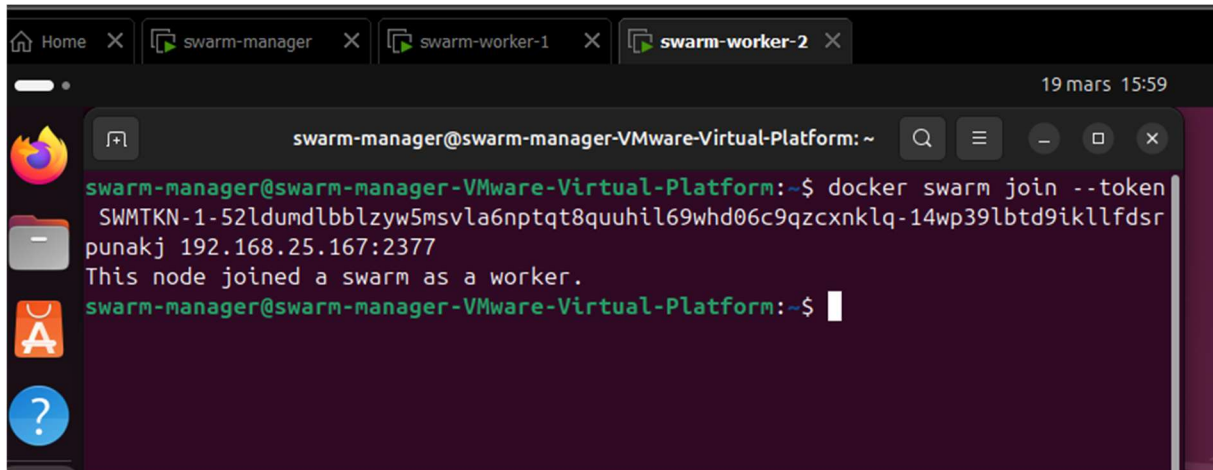
## Étape 4 :

Pour la mise en production simulant un environnement à haute disponibilité, nous avons mis en place un cluster Docker Swarm composé de 3 Machines Virtuelles sous Ubuntu.

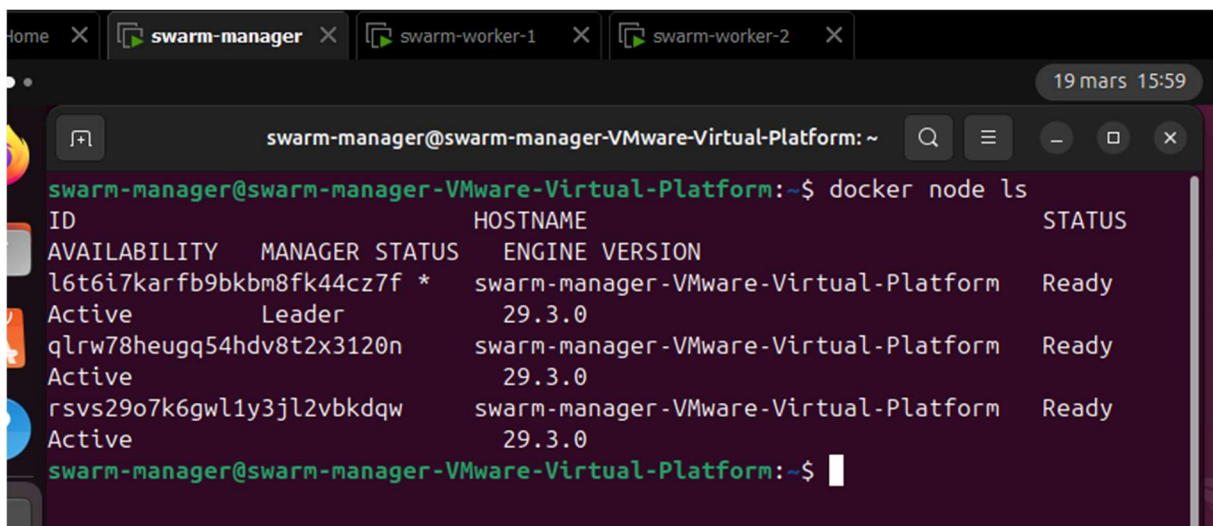
- 1. Mise en place de l'infrastructure :** Nous avons créé et configuré une première VM Ubuntu ("Manager"), sur laquelle nous avons installé Docker. Nous avons ensuite cloné cette machine pour créer nos deux "Workers".



2. **Initialisation du cluster** : Nous avons initialisé le cluster, puis nous avons joint les deux Workers au cluster en utilisant le token généré. La commande `docker node ls` confirme que nos 3 machines sont actives et prêtes.



```
swarm-manager@swarm-manager-VMware-Virtual-Platform: ~$ docker swarm join --token SWMTKN-1-52ldumdlbblzyw5msvla6nptqt8quuhil69whd06c9qzcxnklq-14wp39lbtd9ikllfdsr punakj 192.168.25.167:2377
This node joined a swarm as a worker.
swarm-manager@swarm-manager-VMware-Virtual-Platform:~$
```



```
swarm-manager@swarm-manager-VMware-Virtual-Platform: ~$ docker node ls
ID                                HOSTNAME                                STATUS
AVAILABILITY  MANAGER STATUS  ENGINE VERSION
l6t6i7karfb9bkbm8fk44cz7f *    swarm-manager-VMware-Virtual-Platform Ready
Active        Leader          29.3.0
qlrw78heugq54hdv8t2x3120n      swarm-manager-VMware-Virtual-Platform Ready
Active        Leader          29.3.0
rsvs29o7k6gw1y3jl2vbkdqw       swarm-manager-VMware-Virtual-Platform Ready
Active        Leader          29.3.0
swarm-manager@swarm-manager-VMware-Virtual-Platform:~$
```

3. **Le déploiement de l'application** : Nous avons adapté notre fichier d'orchestration en créant un `docker-compose.swarm.yml`. Les principales modifications incluent :

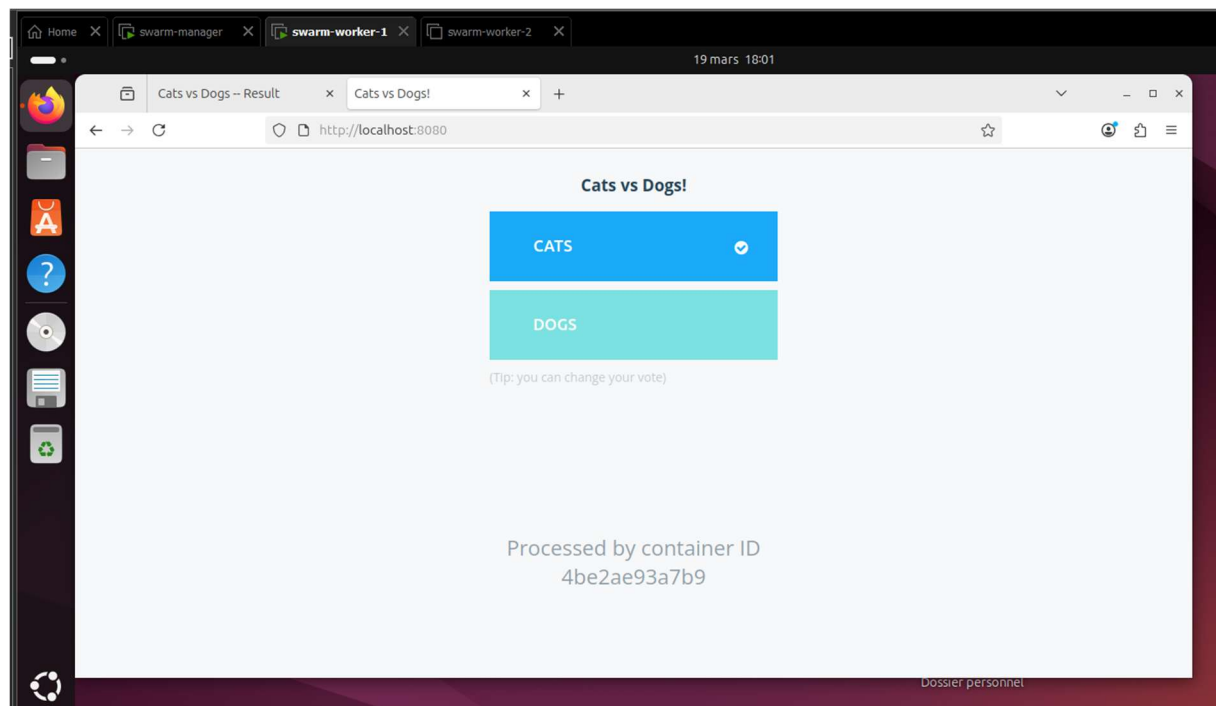
- Le passage des réseaux en driver overlay pour permettre la communication inter-machines.
- L'ajout de la balise `deploy` avec `mode: replicated` et `replicas: 2` pour les applications web (vote et result), garantissant ainsi la haute disponibilité si l'un des nœuds venait à tomber.

- L'ajout de contraintes de placement (node.role == manager) pour forcer les bases de données à rester sur le nœud principal.

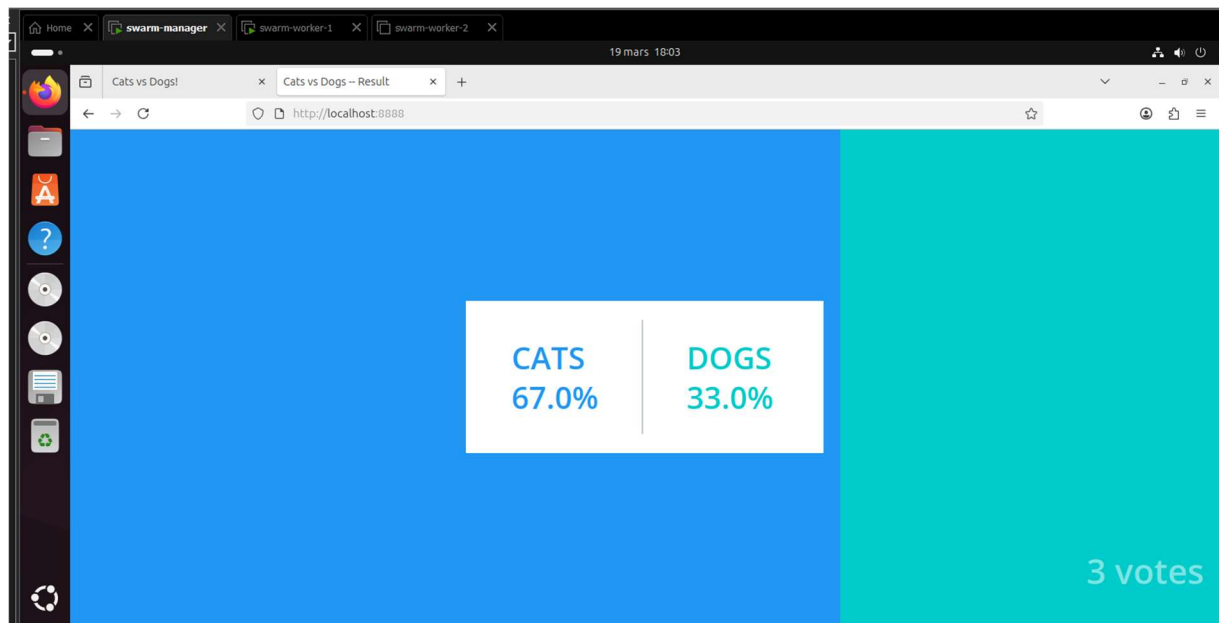
Le déploiement s'est déroulé avec succès via la commande docker stack deploy, avec tous les réplicas correctement distribués et fonctionnels :

```
swarm-manager@swarm-manager-VMware-Virtual-Platform:~$ nano docker-compose.yml
swarm-manager@swarm-manager-VMware-Virtual-Platform:~$ nano docker-compose.yml
swarm-manager@swarm-manager-VMware-Virtual-Platform:~$ docker stack deploy -c docker-compose.yml mon_projet
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network mon_projet_back-tier
Creating network mon_projet_front-tier
Creating service mon_projet_worker
Creating service mon_projet_redis
Creating service mon_projet_db
Creating service mon_projet_vote
Creating service mon_projet_result
swarm-manager@swarm-manager-VMware-Virtual-Platform:~$ docker service ls
ID                NAME                MODE                REPLICAS    IMAGE                                  PORTS
yf9w0u5h4q2v     mon_projet_db       replicated          0/1          postgres:15-alpine
yk0gfbkt0vg7     mon_projet_redis    replicated          0/1          redis:alpine
ssx1rt2iz47      mon_projet_result   replicated          0/2          dockersamples/examplevotingapp_result:latest *:8888->80/tcp
tqhagloeamhc     mon_projet_vote     replicated          0/2          dockersamples/examplevotingapp_vote:latest *:8080->80/tcp
lt1pcquzb433     mon_projet_worker   replicated          0/1          dockersamples/examplevotingapp_worker:latest
```

Le site est accessible depuis l'environnement Swarm, et les votes sont parfaitement comptabilisés à travers le cluster







Le projet est entièrement conteneurisé et distribué avec succès.